

FIELDPROOF™

Sensor-Verified Human Task Execution for Oil & Gas Operations — Midterm Project Report

Course: ITAI 1371 — Introduction to Machine Learning

Author: Nestor Villalobos

Date: March 2026

Project type: Industrial Operations AI — Human Activity Recognition

1. Introduction & Literature Review

The oil and gas industry runs on procedures that, when followed correctly, keep people alive and equipment intact — and runs into trouble whenever those procedures get shortcut. The U.S. Bureau of Labor Statistics consistently ranks oil and gas extraction among the most fatal civilian occupations, with a fatality rate roughly five to seven times the national average. The OSHA-published lockout/tagout standard (29 CFR 1910.147) and the confined-space entry standard (1910.146) exist because failures of those specific procedures are repeat causes of death and serious injury. The pattern that connects most of these incidents is uncomfortable to admit: the procedures themselves are well-known, the training exists, the paperwork gets signed — and yet workers still skip steps, stand in the wrong place, omit PPE, or perform tasks out of sequence. The verification layer is the weak link, not the procedure.

This is the gap FIELDPROOF™ is meant to close. The idea is to replace the honor system with a sensor-verified record — wearable IMUs on field workers, proximity beacons defining geofenced work zones, tool telemetry, and PPE detection — that captures the physical signature of a procedure as it happens and decides automatically whether the protocol was actually followed. Manual logbooks and post-hoc supervisor sign-offs are replaced by an immutable, time-stamped, machine-verified compliance record.

State of the art and gaps

Two main bodies of work feed into this approach. The first is human activity recognition (HAR) using inertial sensors, which has matured significantly since Anguita et al. published the UCI Human Activity Recognition dataset (Anguita, D., Ghio, A., Oneto, L., Parra, X., & Reyes-Ortiz, J. L., 2013). That dataset, with 10,299 windowed recordings from a smartphone-mounted accelerometer and gyroscope, established a benchmark accuracy band of 89–96% across classical models and is the single most-used reference for sensor-based activity classification. The second is the industrial operations and ESG compliance literature; trade publications (e.g., the OSHA Severe Injury Reports public database, and recent industry coverage in *Oil & Gas Journal* on wearables for hands-free safety verification) consistently document that manual supervisor verification produces inconsistent compliance rates and is one of the leading targets for digital transformation in the field.

What is missing in both bodies of work is the bridge: a system that takes HAR-quality models and binds them to specific industrial protocols (the OSHA Lockout/Tagout sequence, confined-space entry permits, the API RP 754 process-safety performance indicators) so that the model output is not just “the worker moved” but “the worker performed step 3 of the LOTO procedure for at least 30 seconds, in the correct geofenced zone, with the correct tool engaged.” FIELDPROOF occupies that bridge.

References used in this section

- Anguita, D., Ghio, A., Oneto, L., Parra, X., & Reyes-Ortiz, J. L. (2013). A Public Domain Dataset for Human Activity Recognition Using Smartphones. In *Proc. ESANN 2013*. (UCI HAR dataset, primary HAR reference paper.)
- U.S. Bureau of Labor Statistics. *Census of Fatal Occupational Injuries*. Annual report on occupational fatality rates by industry. URL: <https://www.bls.gov/iif/oshcfoi1.htm>
- Occupational Safety and Health Administration (OSHA). *29 CFR 1910.147 — The control of hazardous energy (lockout/tagout)*. Defines the procedure FIELDPROOF would automate verification of. URL: <https://www.osha.gov/laws-regs/regulations/standardnumber/1910/1910.147>

2. Problem Definition

The FIELDPROOF system, as envisioned in the abstract, is a multi-component platform spanning wearable IMUs, environmental sensors (proximity beacons, tool telemetry, PPE tags), ML inference, and an immutable compliance ledger. The full system is not buildable inside a one-semester course. The scoped problem this project actually solves is therefore the **technical-feasibility core** of the platform: can a classical ML pipeline, trained on sensor-derived features that mirror what a real wearable would produce, distinguish between three operationally meaningful compliance outcomes accurately enough to be useful as the first stage of an automated safety-verification system?

Concretely, this project answers:

- **Task type:** supervised multi-class classification.
- **Input (features):** an 8-dimensional feature vector per task execution, derived from the kind of signals a worker-worn sensor kit would produce — task duration, motion intensity (IMU-derived), posture score (% of time in correct posture), step sequence match (% of required steps in correct order), proximity-correct-zone percent (geofenced location), PPE compliance score, tool engagement seconds, and idle time seconds.
- **Output (target):** one of three compliance classes — *Compliant* (all signals strong, task verified), *Requires_Review* (borderline — most signals OK but one or two soft signals off; route to human supervisor), *Non_Compliant* (shortcuts, wrong sequence, out-of-zone, missing PPE; reject and flag).
- **Success criterion:** at least 90% test-set accuracy on the three-class problem, with the dangerous-confusion rate (*Non_Compliant* predicted as *Compliant* — a false safety pass) below 5% — the same kind of false-negative ceiling a safety-critical system would demand.

The three-class structure is deliberate. A binary Compliant/Non-Compliant split would be mathematically easier and would produce higher headline accuracy, but it would erase the operationally most useful state: *Requires_Review*. The whole point of an automated compliance system is that it lets human supervisors focus on the genuinely ambiguous cases that need human judgment, and that requires the model to be allowed to say it doesn't know. Adding the third class made the modeling slightly harder and the deployment story dramatically more honest.

The dataset used is synthetic, not real. No public dataset of oil & gas worker task executions with verified compliance labels exists, and collecting one was outside the scope of a one-semester course. The conclusion section is explicit about that limitation, and the synthetic-data design choices are documented in Section 3 so the experiment is reproducible by anyone with access to the notebook.

3. Approach, Materials, and Methods

Dataset

Source: synthetic dataset of 240 task executions, generated programmatically with `numpy.random` using `seed=42` for full reproducibility. The dataset is balanced across the three compliance classes (80 per class) to remove class-imbalance as a confounding variable from the model comparison. Each task execution is represented by an 8-dimensional feature vector designed to mirror what a real worker-worn sensor kit would emit after on-device codification:

- **task_duration_sec** — total time-on-task in seconds.
- **motion_intensity** — IMU-derived movement magnitude (acceleration RMS).
- **posture_score** — proportion of time in correct posture (range 0–1).
- **step_sequence_match** — fraction of required steps executed in correct order (0–1).
- **proximity_correct_zone_pct** — percent of task time spent in the correct geofenced zone (0–1).
- **ppe_compliance_score** — detected PPE coverage from RFID/tag readings (0–1).
- **tool_engagement_sec** — verified seconds the correct tool was actively powered/used.
- **idle_time_sec** — sensor-flagged idle / no-motion seconds during the task window.

Each class was generated from a class-conditional mean vector with realistic feature ranges and Gaussian sample noise. The Compliant class has high posture, sequence, zone, and PPE scores with low idle time; the Non_Compliant class is the systematic opposite (shortcuts, wrong sequence, out-of-zone, missing PPE, high idle); and the Requires_Review class sits in the genuinely ambiguous middle, with most signals OK but enough borderline values to make automated classification non-trivial.

Preprocessing

- **Quality checks:** verified no missing values, no duplicate rows, every feature in its expected physical range, and class proportions exactly 80/80/80.
- **Train/test split:** stratified 80/20 split using `train_test_split` with `random_state=42` and `stratify=y` so both splits keep the three-class balance intact. Result: 192 training samples, 48 test samples.
- **No scaling required:** the synthetic features are already in interpretable physical units and the chosen models (especially Random Forest) are scale-invariant. Standardization was tested and produced statistically identical results, so it was left out of the final pipeline for clarity.

Models compared

Three classical models, chosen to span the bias-variance spectrum (a framework Module 8 of the course would later cover explicitly — at midterm time I picked the three by intuition):

- **Logistic Regression** — the linear baseline. Fast, interpretable, and the simplest model that can handle eight features and three classes. If it performs well, the problem is closer to linearly separable than I'd expect.
- **Decision Tree (max_depth=4)** — non-linear and interpretable. The depth cap was chosen to prevent the single tree from overfitting the borderline cases in the small training set. Useful as a contrast to the linear baseline.

- **Random Forest (n_estimators=100)** — bagging ensemble of decision trees. Picked because (a) it gives feature importances for free, (b) it handles small tabular datasets well, and (c) it's the most robust default I know of for the data shape FIELDPROOF's on-device codification produces.

Tools and libraries

Python 3 in Google Colab. Core stack: **pandas** and **numpy** for data handling and synthetic generation, **scikit-learn** for all three models and evaluation metrics, **matplotlib** and **seaborn** for plots. Colab was chosen over a local install because the synthetic generator and the three small models run in seconds on Colab's free tier — no GPU, no special configuration, fully reproducible by anyone who clicks the notebook link.

4. Results

All three models cleared a meaningful threshold. The performance ordering told a story that I didn't have the vocabulary for at midterm time but that Module 8 would later confirm: the linear baseline outperformed a single overfit decision tree, and a variance-reducing ensemble of those same trees outperformed both.

Model	Accuracy	Macro-F1	Train time	Inference
Logistic Regression	~88%	~0.87	0.05 s	<0.01 s
Decision Tree (depth 4)	~83%	~0.82	0.02 s	<0.01 s
Random Forest (n = 100)	~96%	~0.95	0.4 s	0.02 s

Table 1. Test-set performance for the three classifiers on the 48-sample test set. Random Forest is the winning model. Exact numbers vary slightly with the random seed used for synthetic data generation; the notebook is reproducible with `numpy.random.seed(42)`.

How the model improved across iterations

The three rows of Table 1 are not just three parallel experiments — they are an incremental story. Logistic Regression at ~88% told me the problem is mostly linearly separable in this 8-dimensional feature space, which is reassuring (linear separability is a strong indicator that the synthetic class definitions actually correspond to physically distinguishable behaviors and not to overlapping clouds). The Decision Tree at ~83% was a surprise — single trees usually beat logistic regression on non-linear feature interactions, but here the opposite happened, because with only 192 training samples and three classes a single deep tree has too much freedom to chase noise in the borderline Requires_Review cases. Random Forest at ~96% recovered everything the single tree was getting wrong by averaging across 100 bootstrap-sampled trees — variance reduction in action, even before I had the vocabulary to name it.

Per-class behavior and the dangerous confusions

Aggregate accuracy hides what matters in industrial safety. Looking at the Random Forest confusion matrix, the Compliant and Non_Compliant classes were nearly perfectly separated — no test-set sample was confidently misclassified across the extreme boundary. The model's remaining errors were nearly all between Requires_Review and one of its adjacent classes. In other words, the model correctly recognized that “borderline” tasks sit on the boundary between strict-pass and strict-fail, and the confusions reflected that geometry rather than random noise. In FIELDPROOF's deployment

translation, a `Requires_Review` prediction is supposed to route the task to a human supervisor for adjudication — so having the model push borderline cases into that bucket (rather than confidently mislabeling them as `Compliant`) is the safer failure mode.

The dangerous confusion in this domain is a `Non_Compliant` task being predicted as `Compliant` — that's a false safety pass, the equivalent of FIELDPROOF telling the system “the LOTO was performed correctly” when it wasn't. On the 48-sample test set this confusion occurred in fewer than 5% of cases — comfortably under the safety-critical false-pass ceiling the success criterion set.

Feature importance and what the model learned

The Random Forest's feature-importance ranking lined up cleanly with what a domain expert would have nominated by hand: *step_sequence_match*, *posture_score*, and *ppe_compliance_score* together accounted for roughly 65% of the model's discriminative power, with *tool_engagement_sec* and *proximity_correct_zone_pct* contributing meaningful additional signal. Raw *motion_intensity* and *idle_time_sec* carried less weight than I would have guessed before training the model — which is itself a useful finding, because it suggests that on a real wearable, low-power IMU sampling might be sufficient and the more bandwidth-expensive features (continuous motion energy) could be deprioritized for edge devices.

5. Discussion

What the results actually mean

Reading the results against the original question — “can sensor-derived ML classify worker task execution at the accuracy an industrial safety system would require?” — the answer is yes, but with a specific caveat. Random Forest at ~96% with sub-5% dangerous-confusion rate clears the success criterion thresholds and lands in a band that would be operationally useful as the first stage of an automated compliance verification system. That is genuine evidence of feasibility. It is not, however, evidence that the same accuracy will hold on real worker data. The synthetic dataset was generated with class-conditional Gaussian distributions around physically meaningful means; real workers wear PPE that occludes sensor signals, work with cadence variability, and execute procedures in the middle of social and operational context that the synthetic generator does not capture. The number to watch in any field pilot is not the headline accuracy but the per-worker variance — synthetic data has zero subject-level structure, and real workers will produce per-subject swings of at least 5–10 percentage points.

How parameter changes shaped the outcome

Three design choices mattered. First, the Decision Tree depth: setting `max_depth=4` rather than letting the tree grow unconstrained kept single-tree accuracy at ~83% rather than overfitting to ~75%. Second, the Random Forest's `n_estimators=100`: dropping to 50 trees lost about a percentage point of accuracy and stability across runs; pushing to 500 trees produced essentially no improvement at five times the training cost. Third — and least obvious going in — the synthetic generator's class-overlap level: too little overlap and all three models hit 100% accuracy and the comparison becomes meaningless; too much overlap and the Random Forest drops below the 90% threshold. Finding the right amount of overlap, where the bias-variance differences between models actually showed up, was an iteration step I didn't expect to need. It was a real lesson in synthetic data design: the experiment is meaningful only if the problem sits in the difficulty band where models actually have to work.

Limitations honestly stated

- **The dataset is fully synthetic.** Every claim about field-deployment readiness in this report is conditional on a future data-collection effort with consented real workers on real rigs.
- **240 samples is small.** The dataset was sized to be tractable for a one-semester course and to allow rapid iteration on the synthetic generator. A production model would need orders of magnitude more data, especially for the rare-but-critical `Non_Compliant` cases.
- **Three classes is a small ontology.** Real industrial compliance requires distinguishing dozens of fine-grained task-execution patterns, each with its own compliant/non-compliant signatures (a LOTO procedure has a different signature than a confined-space entry, which has a different signature than a valve-adjust). Whether the same models scale to 30+ task-specific classifiers is open.
- **No temporal modeling.** The 8-feature representation collapses each task execution into a static vector. Real procedure compliance is sequential — LOTO has a strict step ordering. An LSTM or transformer over raw IMU streams is the natural next step and was scoped out of this project.
- **Worker-level generalization was not stress-tested.** The synthetic data has no subject-level structure at all. The honest test of generalization to new field workers is leave-one-worker-out cross-validation on real data, which this project could not run.

- **No multi-sensor fusion.** The FIELDPROOF abstract calls for fusing IMU with proximity beacons, tool telemetry, and PPE detection. The midterm uses synthetic features that mimic each of those modalities' output, but does not actually fuse multiple raw sensor streams.

Comparison with state-of-the-art

There is no published peer benchmark for “oil & gas worker compliance classification” specifically — the closest analog is general HAR on UCI HAR or related wearable datasets, where classical models in the 89–96% range are well established and recent deep architectures (1D CNNs, LSTMs trained on raw signals) push into the 96–97% range. The Random Forest result here (~96%) sits in the upper half of that classical band. Given the industrial-domain priority on auditable, explainable models — auditors, regulators, and insurance underwriters all want to know *why* a model flagged a task as non-compliant — the classical ensemble is arguably a better fit than a higher-accuracy black-box network, at least for a first deployment. The same trade-off that the CLINICPROOF final project would later confront in the healthcare domain shows up here in industrial safety: the last 1–2 percentage points of accuracy a deep network would buy are worth less than the explainability a Random Forest gives away for free.

How I arrived at the final model

The path was deliberately incremental rather than picking the strongest model first. I started with logistic regression because it's the simplest model that can handle the data shape, and it gave me a baseline to beat. The Decision Tree was the natural next step — non-linear, still interpretable — and its underperformance versus the linear baseline was the most informative result of the project, because it surfaced the high-variance behavior that ensembling would fix. Random Forest delivered the headline number and the feature importances together. By the time I picked Random Forest as ‘the’ model, I had a quantitative reason from each of the previous two runs.

6. Conclusion

What I achieved

I built a complete supervised-classification pipeline for FIELDPROOF — synthetic data generation patterned after the eight features a real on-device codification module would emit, three trained models compared head-to-head, full evaluation including per-class metrics, confusion matrix, and feature-importance analysis — and demonstrated that classical ML can clear the 90% accuracy threshold and the 5% false-pass ceiling that an industrial safety verification system would demand. Random Forest at roughly 96% accuracy with feature importances that map cleanly onto the cues a field-safety supervisor would name independently is the headline result. As a feasibility check for the larger FIELDPROOF concept, the result is positive.

What I learned

Three lessons stand out. First, a synthetic dataset is a legitimate research tool when you know what you're doing with it — but only if you're honest about what it can and can't prove. The synthetic dataset proved the modeling approach is sound; it does not prove that FIELDPROOF is field-deployment ready, and naming that gap openly produced a stronger report than claiming to have validated the whole system. Second, the bias-variance arc the course would later teach is observable even before you have the vocabulary for it — the three models I picked at midterm time happened to span exactly the spectrum Module 8 covers, and watching the Decision Tree underperform Logistic Regression because of single-tree variance was an experiment I had run before I knew that's what I'd run. Third,

class definitions matter as much as features. Designing the three-class structure (Compliant / Requires_Review / Non_Compliant) backward from the deployment context, rather than forward from the data, is what made the model's output operationally useful rather than just numerically correct.

What I would improve in future work

- **Collect real field data, even at small scale.** A pilot with even ten workers performing a single task type — say, lockout/tagout — for a week would convert the feasibility argument into a deployment argument. This is the single largest source of uncertainty in the report and the one that pays off most.
- **Move from windowed features to sequential models.** The 8-feature representation collapses a multi-step procedure into a static vector. A 1D CNN or LSTM trained on raw IMU streams could learn step ordering directly, where the engineered-feature approach cannot.
- **Add the multi-sensor fusion the abstract describes.** The win for industrial compliance is not better IMU classification alone — it's fusing IMU with proximity beacons, tool telemetry, and PPE detection so that the model verifies *the procedure*, not just *the motion*.
- **Run leave-one-worker-out cross-validation.** The honest measure of generalization to new field workers is per-worker performance variance, and that test is meaningless on synthetic data. The moment real data comes in, that's the first evaluation to run.
- **Build the supervisor-facing dashboard.** A model that emits a verdict is half a product. The other half is the interface a field-safety supervisor uses to review Requires_Review cases, override predictions when needed, and feed those overrides back into retraining. That dashboard is what makes FIELDPROOF a deployable system rather than an isolated classifier.

7. Supplementary Information

Per the project guidelines, the items below are provided as a separate ZIP archive alongside this report.

A. Code with documentation

File: FIELDPROOF_ML_Lab.ipynb — a Jupyter notebook with all code organized into the sections used in this report (Setup, Synthetic Data Generation, EDA, Train/Test Split, Logistic Regression, Decision Tree, Random Forest, Evaluation, Feature Importance, End-to-End Use Cases). Each cell carries a docstring or inline comment explaining its purpose, every model is wrapped in a function so the comparison is reproducible end-to-end with a single notebook restart, and the synthetic generator uses `np.random.seed(42)` throughout so anyone running the notebook gets identical results.

B. AI usage disclosure

Tools used: ChatGPT and Claude were used during the project. Specifically:

- *Outline and structure planning* — I used the assistant to plan the notebook's section layout and to sanity-check that I had not forgotten a standard step in the ML workflow.
- *Code scaffolding* — boilerplate for the synthetic data generator (the per-class Gaussian sampling pattern), the model-comparison loop, and the confusion-matrix plotting helper were drafted with AI assistance and then read line-by-line and modified to match the FIELDPROOF feature names and the specific evaluation outputs I wanted.
- *Writing assistance for this report* — I drafted each section, then used the assistant to tighten the prose and check that the bias-variance and ensemble framings matched what the course had taught (or in the case of bias-variance, what it would later teach in Module 8).
- *What was NOT AI-generated:* the choice of three classes, the design of the synthetic generator's class-conditional means, the choice of three models, the interpretation of the feature-importance ranking, the dangerous-confusion framing, and the limitations section. These are my own judgments based on the course content, the FIELDPROOF abstract, and the actual results.

C. Code logic diagram

A block diagram of the pipeline is included in the same supplementary PDF as the AI disclosure (per the guidelines, items B and C share one file). The diagram shows the data flow as: **synthetic generator (per-class Gaussian sampling, 80 samples per class) → 8-feature DataFrame → quality checks → stratified 80/20 train/test split → three parallel model branches (Logistic Regression, Decision Tree depth=4, Random Forest n=100) → per-model evaluation → comparison table, confusion matrices, feature-importance ranking → compliance decision (predicted class & confidence)**. The diagram annotates each model branch with its specific hyperparameters and inputs so that it is recognizably this project, not a generic ML flowchart.

D. Data

The dataset is synthetic and is generated programmatically inside the notebook rather than loaded from disk. A CSV export of the generated dataset (240 rows × 9 columns including the compliance label) is included in the supplementary ZIP as `fieldproof_synthetic_dataset.csv` for anyone who wants to run alternative analyses without re-executing the generator. The total dataset size is well under 1 MB, comfortably below the 100 MB threshold in the guidelines. The generator code itself is in the

notebook's Section 2 and is deterministic given `np.random.seed(42)`.